

GRPO proof-of-life on Qwen3.6-27B

Faisal · July 16, 2026 · the plan we execute the moment Michael sends the model API key + SSH. Every benchmark fact below was verified against source code and papers over July 15–16.

This is the dress rehearsal before we point RL at our own environment. It answers one question — **can we take Qwen3.6-27B and, with GRPO, move a program-scored slide benchmark by +2–3% on a measurement that can't be faked?** If yes, the training plumbing, reward mapping, and honest-measurement method are all proven, and Phase 2 (powerbench) starts on July 28 with the risky parts already de-risked.

1. What this proves — and what it does not

GRPO (Group Relative Policy Optimization) is our training method: the model attempts each task several times, a grader scores every attempt, and the model is nudged toward whatever scored higher. This experiment proves three things end-to-end:

- the training loop runs on the 27B without exploding (config, memory, throughput);
- a grader's numbers can be turned into a learning signal (the *reward mapping*) that actually raises the score;
- we can measure the gain on a contamination-proof held-out set — i.e., the improvement is real, not memorized.

It deliberately does **not** prove template-conformance or the multi-turn agent loop — those are powerbench's job. SlidesBench is single-shot slide-code generation, so it isolates the plumbing cleanly. Treat a pass here as "the machine works," not "the product works."

2. Hypotheses (stated so they can fail)

H1 — trainable zone. At baseline, Qwen3.6-27B scores in a workable middle band on SlidesBench's reference-based metrics — high enough to have signal, low enough to have headroom. *Fails if* it already scores near the ceiling (nothing to learn) or near zero (no gradient to climb).

H2 — the gain is real. After GRPO, the 27B improves by $\geq +2-3\%$ on the *perturbed* held-out set (defined in §5), not just the official one. *Fails if* gains appear only on the official set (= memorization) or nowhere.

H3 — eval sanity. The 27B's baseline lands sensibly against SlidesBench's *published* leaderboard (GPT-4o, the paper's own trained model) — not absurdly high, not on the floor. This confirms our scoring pipeline is wired correctly before we trust any training delta. It costs nothing — a comparison to numbers already in the paper, not a second model we have to run.

3. Model & input mode

MODEL	ROLE	SERVING	INPUT MODE
Qwen3.6-27B	Train target — the only model we run	vLLM on Michael's GPU box (needs SSH)	With-image (native)

One model, on purpose. A proof-of-life needs exactly the model we intend to train — nothing else. We are *not* running a big second model as a yardstick: it would be too large and expensive to be worth it here, and we don't need one to prove the loop works. The sanity check comes from SlidesBench's own published leaderboard instead (H3), which costs nothing.

Input mode: SlidesBench ships three instruction variants — with-image, high-level, and no-image. We run the 27B on the **with-image** variant: it's vision-capable, and this mirrors how the real pptx skill's render→look→fix loop actually works. We'll also record the `no_image` score as a harder reference point.

4. Benchmark & metrics — what we score, what we throw out

Target: **SlidesBench** (from the AutoPresent paper). The model writes Python that builds a `.pptx` ; we execute it and score the result. Verified by reading `evaluate/page_eval.py` directly — these four metrics are deterministic `.pptx` -vs- `.pptx` comparisons with no AI in the loop:

METRIC	WHAT IT CHECKS	USE?
Execution success	Did the generated code run and produce a valid pptx?	Yes — reward gate
Text similarity	Does the slide text match the reference?	Yes
Color similarity (CIEDE2000)	Are colors within perceptual distance of the reference?	Yes
Position similarity	Are elements placed where the reference has them?	Yes
Element / block match	Are the right elements present (max-matching)?	Yes
Reference-free "design quality"	Overall aesthetic judgment	No — GPT-4o-judged, not deterministic
CLIP image-similarity	Visual embedding match	No — model-based, hackable

We score on the **reference-based deterministic family only**. Dropping the two model-based metrics is what keeps the reward un-gameable and the +2–3% a fact rather than an opinion.

5. Contamination controls — the honest-measurement method

SlidesBench has been public since January 2025, ~15 months before the 27B's April 2026 release, and Alibaba publishes no data cutoff — so the model may have seen the test answers. We neutralize this two ways. (Reminder: contamination only poisons the *measurement*; training on seen data is fine — it's what fine-tuning is.)

5a. Memorization probe (~30 min, run first)

Feed each model half of a test instruction and see if it completes the reference verbatim; and check whether baseline scores sit implausibly above the published state of the art. Either signal ⇒ the official test set is contaminated and we lean entirely on the perturbed set below.

5b. Perturbed held-out set (the real eval, ~½ day to build)

Take ~50 test tasks and programmatically alter each reference slide, then *regenerate the instruction from the altered slide* so the task stays self-consistent. Memorized answers are now wrong by construction.

Operators:

- **Color** — shift each themed color by a controlled perceptual distance (swap palette entry / rotate hue).
- **Text** — replace titles/body with novel strings, same structure.
- **Position** — translate element bounding boxes.
- **Element count** — add or drop a required element.

Close the loop with SlidesBench's own `create_dataset.py` / `seed_instruction.py` — perturb the reference, regenerate its instruction, re-score. This is also how we'll **mint our own training tasks** (the official 7k download is broken; unofficial mirrors are unknown fidelity). **Pre-check:** confirm the deterministic metrics don't produce degenerate scores under our operators before trusting the numbers — the one open question the verification pass flagged. Report official *and* perturbed scores side by side; if they diverge, the perturbed number is the truth.

6. Training setup

- **Framework:** verl (GRPO is first-class; the Qwen3-MoE family is its worked example). **Method:** LoRA-GRPO — train small add-on layers, not all 27B weights, so it fits the box and the window.
- **Group size:** 8 rollouts per prompt to start (matches our $k=8$ convention and AeSlides' setup), advantage = reward minus group mean.
- **Starting knobs to tune in Gate 1:** LoRA rank ~ 32 , one epoch over the minted train set, response cap sized to a slide program ($\sim 4\text{--}8\text{k}$ tokens). These are starting points, not commitments — Gate 1 exists to tune them.

Reward mapping (Q2 — I draft, Cara checks for hackability)

Turn the metrics into one number per attempt, with execution as a hard gate so broken code earns nothing:

```
reward = 0 if code fails to execute or yields no valid pptx
reward = wtext·text + wcolor·color + wpos·position + welem·element otherwise (weights sum to 1, each term in [0,1])
```

Anti-hacking notes: give `element` real weight so the model can't win with an empty slide that trivially matches on color; keep weights blunt (no per-task tuning that invites overfitting). This mirrors powerbench's eventual mapping (per-check booleans $\rightarrow$ weighted scalar), so whatever we learn about

weights and hackability here transfers straight to Phase 2 — that's a deliberate design choice, not a coincidence.

7. Runbook — gated on access, not the calendar

PHASE 0 — NOW NO ACCESS NEEDED

Harness ready (already cloned + venv built). Write the perturbation operators and the reward function; template the verl GRPO config; wire cost tracking + resume-on-failure into the rollout orchestration. Deliverable: a dry-run that scores a hand-made pptx against a reference locally.

PHASE 1 — WHEN MICHAEL SENDS THE API KEY WAITING

Memorization probe (§5a) on the 27B. Baseline eval on the 27B (with-image + no-image), official + perturbed sets, compared against SlidesBench's published leaderboard. **Answers H1 and H3.** This is also powerbench Gate 2 input: does the 27B land in the workable band?

PHASE 2 — WHEN MICHAEL SENDS SSH TO THE GPU BOX WAITING

Stand up vLLM serving for the 27B. Run **Gate 1** (tiny verl step — confirm the Gated-DeltaNet architecture trains; likely per docs, but verified hands-on). Then the GRPO proof-of-life run on the minted train set. **Answers H2.**

PHASE 3 — READ THE RESULT

Compare pre- vs post-training on the perturbed set. Report the curve honestly (a real +1% beats a fabricated +5%). Hand the reward-mapping + hackability findings to Cara for the powerbench mapping.

Gate 1 (trainability): tiny verl run completes a step on the 27B → proceed. Fails → fall back to Kimi K2.7-Code on Fireworks RFT, no fighting it.

Gate 2 (band entry): 27B lands in the workable middle band at baseline → proceed to train. Scores near-zero everywhere → too weak, fall back. Saturates → switch to a harder eval slice.

8. Decision rules — what each outcome means

RESULT	READING	ACTION
≥ +2–3% on perturbed set	Proof-of-life achieved	Plumbing proven; carry reward mapping into powerbench Phase 2

Gain on official set only, not perturbed	Memorization, not learning	Trust perturbed number; investigate reward before Phase 2
No gain anywhere	Plumbing or reward broken	Debug loop on the 235B sibling; don't burn the window on the big model
Gate 1 fails (can't train 27B)	Architecture unsupported	Fireworks RFT + Kimi K2.7-Code within 24h
27B baseline near published SOTA	Pipeline wired right	Trust the deltas; proceed

9. Risks & mitigations

- **SlidesBench repo is unmaintained** (no commits since May 2025, 8 open issues incl. a quickstart crash). → We own the fixes; harness already runs in our venv; mint tasks with the repo's own scripts rather than the broken download.
- **Perturbation could break the metrics** (degenerate scores). → Pre-check metric stability on a handful of perturbed tasks before building all 50 (§5b).
- **Gated-DeltaNet unsupported in verl.** → Gate 1 catches it in 30 minutes; Fireworks fallback ready.
- **Single-turn $\neq$ powerbench's multi-turn.** → Acknowledged scope limit; the multi-turn path is exercised by the powerbench smoke test, not here.
- **Contamination unmeasurable precisely** (cutoff unpublished). → The perturbed set sidesteps it entirely; we don't need the cutoff.

10. Compute, cost & what we need from Michael

Compute: Phase 1 is hosted-API inference for the 27B (small — hundreds of short generations; a few dollars on credits). Phase 2 GRPO is a single 8-GPU node (the 27B fits with room for 8 parallel rollouts). No multi-node cluster — that's the whole reason we're training the 27B and nothing bigger.

Blocking on Michael:

- **needed** Model **API key** → unblocks Phase 1 (baseline + probe).
- **needed** **SSH** to the GPU box → unblocks Phase 2 (serving + training).
- GitHub invite to `powerbench` (still 404s for my account) and bot access to `#pptx-benchmarks` to reconcile against the team's own list.

Everything in Phase 0 proceeds now, without any of the above.

Companion to *model-recommendation.pdf* (train the 27B) and *benchmark-guide.pdf* (why SlidesBench + perturbation).
Sources verified July 15–16, 2026: AutoPresent/SlidesBench arXiv 2501.00912 + repo code (page_eval.py, create_dataset.py read directly) · verl GRPO docs + Qwen3-MoE best-practices · AeSlides arXiv 2604.22840 + repo (train set / code confirmed withheld — demoted to methods reference) · Qwen3.6-27B card/blog (no cutoff or decontamination statement) · Fireworks RFT docs (Kimi K2 fallback). Three adversarial verification passes, 200+ sources fetched.